

ບົດທີ 7 ການຄວບຄຸມເວີຊັນດ້ວຍ Git

Version Control with Git

🕒 ປະຫວັດການປ່ຽນແປງ

👥 ການຮ່ວມມື

🌿 Branching

📁 Backup & Restore

ເນື້ອໃນຫຍໍ້

-  1 ເປັນຫຍັງຕ້ອງມີການຄວບຄຸມເວີຊັນ?
-  2 ຄຳສັ່ງ Git ພື້ນຖານທີ່ທຸກຄົນຕ້ອງໃຊ້
-  3 ແຟລດຟອມການເຮັດວຽກຮ່ວມກັນ: GitHub/GitLab
-  4 ແນວທາງການເຮັດວຽກຂອງ Git (Git Workflow Strategies)

ເປັນຫຍັງຕ້ອງມີການຄວບຄຸມເວີຊັນ?

ສະຖານະການຝັນຮ້າຍ (The Nightmare Scenario)

▲ ການຂຽນທັບກັນ (Overwriting)

ຫຼາຍຄົນແກ້ໄຂໄຟລ໌ດຽວກັນ → ວຽກຂອງຄົນອື່ນຖືກຂຽນທັບ

🕒 ບໍ່ຮູ້ວ່າໃຜແກ້ຫຍັງ (Lack of History)

ເກີດ Bug → ບໍ່ຮູ້ "ໃຜ" ເປັນຄົນແກ້, "ເມື່ອໃດ", ແລະ "ເພາະຫຍັງ"

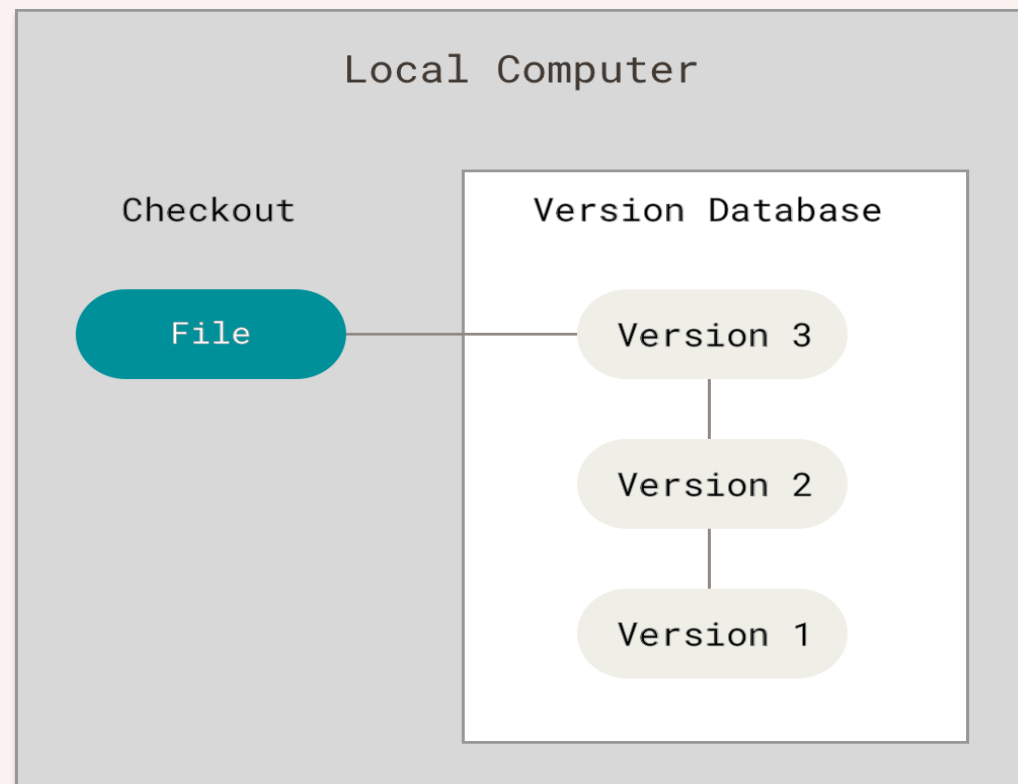
🕒 ການກັບຄືນໄປເວີຊັນເກົ່າ (Reverting Changes)

ເຮັດໂຄດຟັງ → ຢາກກັບໄປເວີຊັນທີ່ໃຊ້ງານໄດ້ດີ → ບໍ່ມີການບັນທຶກ

▲ ການທົດລອງ Feature ໃໝ່ (Experimentation)

ຢາກລອງວິທີໃໝ່ → ຢ້ານກະທົບກັບໂຄດຫຼັກທີ່ກຳລັງໃຊ້ຢູ່

ຄວາມວຸ້ນວາຍ ໃນການຈັດການໄຟລ໌ ໂດຍບໍ່ມີ VCS



ບໍ່ມີການຄວບຄຸມເວີຊັນ = ບໍ່ມີລະບົບທີ່ຈັດການປະຫວັດການປ່ຽນແປງ

Version Control System (VCS) ពិជ្ជមាន

⚙️ ຄືຊອບແວທີ່ຊ່ວຍຕິດຕາມ ແລະ ຈັດການ

ຕິດຕາມ **ທຸກໆການປ່ຽນແປງ** ທີ່ເກີດຂຶ້ນກັບຊຸດຂອງໄຟລ໌
(Source Code)

🕒 ເຄື່ອງຈັກເວລາ (Time Machine)

ສ້າງ "ຈຸດປັນທຶກ" (Snapshot ຫຼື Commit) ໄວ້ໄດ້ທຸກເວລາ

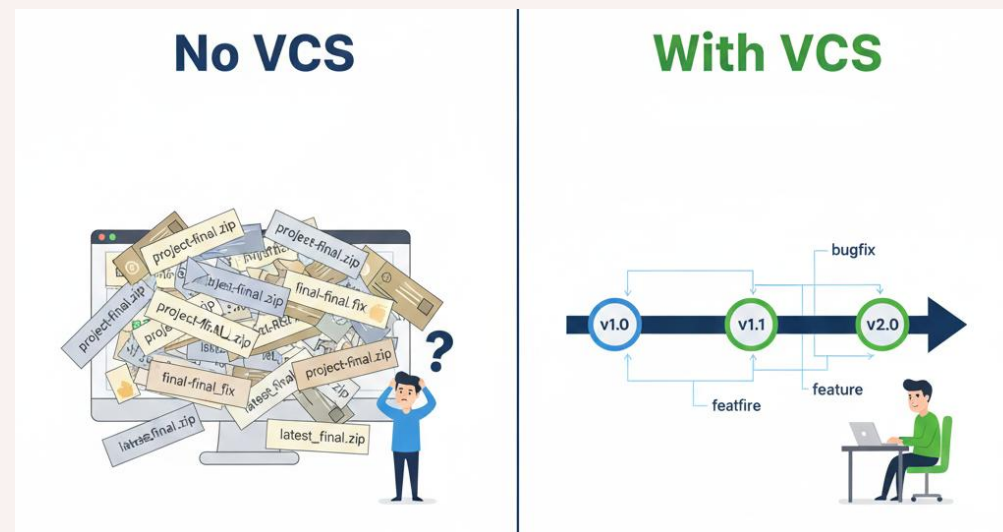
 ຂໍ້ມູນທີ່ບັນທຶກ

ເກັບສະຖານະຂອງໂຄດທັງໝົດ + ບັນທຶກ "ໃຜ" + "ເມື່ອ
ໃດ" + "ເພາະຫຍັງ"

🕒 ການກັບໄປເບິ່ງປະຫວັດ

ສາມາດກັບໄປເບິ່ງໂຄດໃນເວີຊັ້ນທີ່ຜ່ານມາໄດ້ທຸກເວລາ

ການເຮັດວຽກແບບບໍ່ມີ ແລະ ມີ VCS



VCS ຊ່ວຍປ່ຽນຄວາມວຸ່ນວາຍໃນການຈັດການໄຟລ໌ ໃຫ້
ກາຍເປັນປະຫວັດການປ່ຽນແປງທີ່ເປັນລະບຽບ

ປະໂຫຍດຫຼັກຂອງການໃຊ້ VCS

👥 ການຮ່ວມມື (Collaboration)

ຄົນຫຼາຍຄົນສາມາດເຮັດວຽກໃນໂຄງການດຽວກັນ, ໃນຟາຍດຽວກັນ, ໄດ້ຢ່າງປອດໄພ

🕒 ການເກັບປະຫວັດ (History)

ທຸກໆການປ່ຽນແປງຖືກບັນທຶກ → ຮູ້ວ່າ **ໃຜ** ເປັນຄົນຂຽນ, **ເມື່ອໃດ**, ແລະ **ເພາະຫຍັງ**

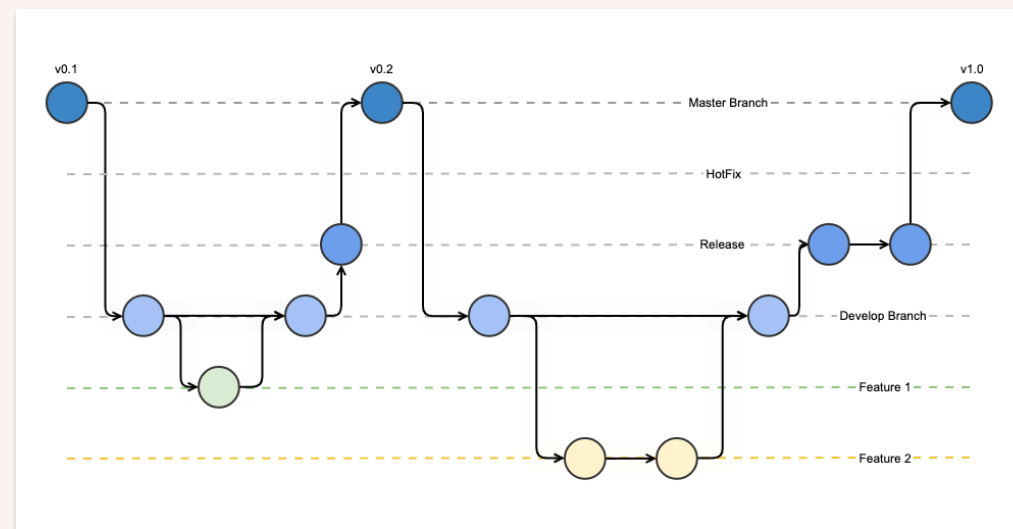
💾 ການສຳຮອງຂໍ້ມູນ ແລະ ການກູ້ຄືນ (Backup & Restore)

ທຸກ "ຈຸດບັນທຶກ" ຄືການສຳຮອງຂໍ້ມູນແບບສົມບູນ

🌿 ການແຕກກຶ່ງ (Branching)

ສ້າງ "ກຶ່ງ" ເພື່ອທົດລອງ Feature ໃໝ່ ຫຼື ແກ້ໄຂ Bug ໂດຍບໍ່ກະທົບໂຄດຫຼັກ

ແນວຄິດຂອງ Branching



Branching ຊ່ວຍໃຫ້ສາມາດພັດທະນາ Feature ໃໝ່ໄດ້ຢ່າງປອດໄພ ໂດຍບໍ່ສົ່ງຜົນກະທົບຕໍ່ໂຄດຫຼັກ

ຄໍາສັ່ງ Git ພື້ນຖານ (ສ່ວນທີ 1)

✚ `git init` `git init`

ສ້າງ Repository ໃໝ່ທີ່ວ່າງເປົ່າ, ສ້າງ Folder ລັບຊື່ວ່າ `.git` ຂຶ້ນມາ

✚ `git add` `git add.`

ເພີ່ມໄຟລ໌ເຂົ້າ **Staging Area**, ບອກ Git ວ່າຕ້ອງການ ຕິດຕາມການປ່ຽນແປງ

📁 `git commit` `git commit -m "message"`

ບັນທຶກ **Snapshot** ຫຼື **"ຈຸດບັນທຶກ"** ຂອງທຸກໄຟລ໌ທີ່ຢູ່ ໃນ Staging Area

🔴 **ການເຮັດວຽກພື້ນຖານ**

ວຽກຂອງເຮົາຈະເລີ່ມຈາກ Working Directory → Staging Area → Repository

The Three States of Git



ວຽກຂອງເຮົາຈະເລີ່ມຈາກ Working Directory, ຖືກເພີ່ມ ເຂົ້າ Staging Area, ແລະ ຖືກບັນທຶກລົງ Repository

ຄໍາສັ່ງ Git ພື້ນຖານ (ສ່ວນທີ 2)

⬇ `git clone` `git clone URL`

ສ້າງ "ສໍາເນົາ" ຂອງ Repository ຈາກ Server ລົງມາ
ໃນເຄື່ອງຂອງເຮົາ

⬆ `git push` `git push origin main`

ສົ່ງ "ຈຸດປັບທຶກ" ທັງໝົດທີ່ເຮົາສ້າງຂຶ້ນ ຂຶ້ນໄປເທິງ Server

☁ `git pull` `git pull origin main`

ດຶງ "ຈຸດປັບທຶກ" ໃໝ່ໆ ຈາກ Server ລົງມາລວມ ກັບ
ໂຄງການໃນເຄື່ອງຂອງເຮົາ

⚡ `git branch` `git checkout -b name`

ສ້າງ, ລຶບ, ຫຼື ສະແດງລາຍຊື່ "ກິ່ງງ່າ" ທັງໝົດ

↕ `git merge` `git merge branch-name`

ລວມເອົາປະຫວັດການປ່ຽນແປງຈາກ Branch ໜຶ່ງ
ເຂົ້າມາໃນ Branch ປັດຈຸບັນ

ການເຮັດວຽກຮ່ວມກັບ Remote Repository



Git ຊ່ວຍໃຫ້ຫຼາຍຄົນສາມາດເຮັດວຽກຮ່ວມກັນໄດ້
ຜ່ານ Remote Repository

GitHub/GitLab ແມ່ນຫຍັງ?

- ☁ **ບໍລິການເວັບໄຊ (Web-based hosting service)**
ຊ່ວຍໃຫ້ເຮົາສາມາດເກັບ **Remote Git Repositories** ໄວ້ເທິງ Cloud ໄດ້
- ⚙ **ບໍ່ແມ່ນ "Git" ແຕ່ເປັນ "ບໍລິການທີ່ສ້າງຂຶ້ນອ້ອມຮອບ Git"**
ເປັນແພລຕຟອມທີ່ເພີ່ມຄວາມສາມາດໃຫ້ກັບ Git

Git
"ລະບົບ" ທີ່ເຮັດວຽກ
ໃນເຄື່ອງຂອງເຮົາ

GitHub/GitLab
"Social Media
ສໍາລັບນັກພັດທະນາ"

- 🌐 **ບ່ອນທີ່ເຮົາ Git Repository ໄປ "ອອນລາຍ"**
ໃຫ້ຄົນອື່ນສາມາດເຫັນ, ດາວໂຫຼດ, ແລະ ຮ່ວມມືໃນ
ໂຄງການຂອງເຮົາໄດ້

GitHub ແລະ GitLab



ທັງສອງເປັນແພລຕຟອມທີ່ນິຍົມສໍາລັບການຈັດການ
ໂຄດແລະການຮ່ວມມືຂອງທີມງານ

ຫຼາຍກວ່າພຽງແຕ່ບ່ອນເກັບໂຄດ

GitHub/GitLab ບໍ່ໄດ້ເຮັດໜ້າທີ່ພຽງແຕ່ເກັບໂຄດ, ແຕ່ໄດ້ເພີ່ມ Feature ຕ່າງໆ ເຂົ້າມາອ້ອມຮອບ Git ເພື່ອສ້າງເປັນ "ລະບົບນິເວດ" ຄົບວົງຈອນສໍາລັບການພັດທະນາຊອບແວ

<> ການເກັບໂຄດ

ເປັນບ່ອນເກັບ Remote Repository

📄 ການທົບທວນໂຄດ

ລະບົບ Pull/Merge Request

🔍 ການຕິດຕາມບັນຫາ

ສ້າງ, ມອບໝາຍ, ແລະ ຕິດຕາມ Bug

📖 ເອກະສານ

ລະບົບ Wiki ສໍາລັບ ເອກະສານຂອງໂຄງການ

⚙️ CI/CD

ເຄື່ອງມືອັດຕະໂນມັດ Test ແລະ Deploy

ລະບົບນິເວດຂອງ GitHub/GitLab

HOW TO CHOOSE THE RIGHT ONE



GitHub/GitLab ເປັນຫຼາຍກວ່າບ່ອນເກັບໂຄດ, ແຕ່ເປັນລະບົບນິເວດຄົບວົງຈອນສໍາລັບການຮ່ວມມື

Workflow ທີ່ສໍາຄັນທີ່ສຸດ: Pull Request

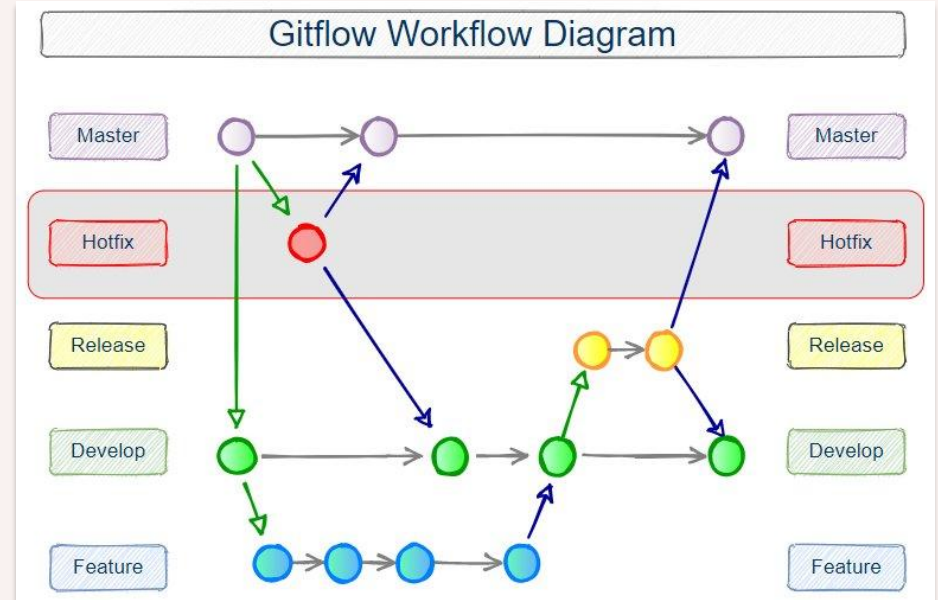
ຂະບວນການເຮັດວຽກທີ່ເປັນ**ຫົວໃຈ**ຂອງການຮ່ວມມືເທິງ
GitHub/GitLab

- 1 ສ້າງ Branch**
ສ້າງ Branch ໃໝ່ຈາກ main branch
- 2 ລົງມືເຮັດວຽກ**
ຂຽນໂຄດ, ແກ້ໄຂ, ແລະ commit ການປ່ຽນແປງ
- 3 Push Branch**
ສົ່ງ Branch ຂອງທ່ານຂຶ້ນໄປເທິງ GitHub/GitLab
- 4 ເປີດ Pull Request**
ກົດ "Open a Pull Request" ເພື່ອຂໍໃຫ້ລວມເຂົ້າ main branch
- 5 ທົບທວນ ແລະ ສືບທະນາ**
ທົບທວນເຂົ້າມາ Review, ຂຽນຄອມເມັນ, ແລະ CI/CD ເຮັດວຽກ
- 6 ອະນຸມັດ ແລະ ລວມ**
ຫົວໜ້າທີມກົດ Approve ແລະ Merge Pull Request

💡 ຕົວຢ່າງການແກ້ Bug #123

ສ້າງ Branch **fix/bug-123** → ແກ້ໄຂໂຄດ → Push →
ເປີດ PR → Review → Merge

The Pull Request Workflow



ຂະບວນການ Pull Request ເປັນຫົວໃຈຂອງການຮ່ວມມື,
ຮັບປະກັນວ່າທຸກການປ່ຽນແປງຈະຜ່ານການກວດສອບ
ກ່ອນລວມເຂົ້າໂຄດຫຼັກ

Git Workflow Strategies

Git Workflow ຄືຊຸດຂອງຄໍາແນະນໍາ ຫຼື ບຸດທະສາດທີ່ກໍານົດວ່າທີມງານຄວນຈະໃຊ້ Git ແນວໃດ

- ✓ **ການເຮັດວຽກເປັນລະບຽບ**
ກໍານົດຂັ້ນຕອນການເຮັດວຽກທີ່ຊັດເຈນ
- 🔧 **ຫຼຸດຜ່ອນຂໍ້ຂັດແຍ່ງ**
ຫຼຸດຜ່ອນການຂັດແຍ່ງໃນການລວມໂຄດ (Merge Conflicts)
- 🕒 **ປະຫວັດທີ່ສະອາດ**
ເຮັດໃຫ້ປະຫວັດຂອງໂຄງການ (Git History) ສະອາດ ແລະ ຕິດຕາມງ່າຍ

2 Workflows ທີ່ເປັນທີ່ນິຍົມທີ່ສຸດ

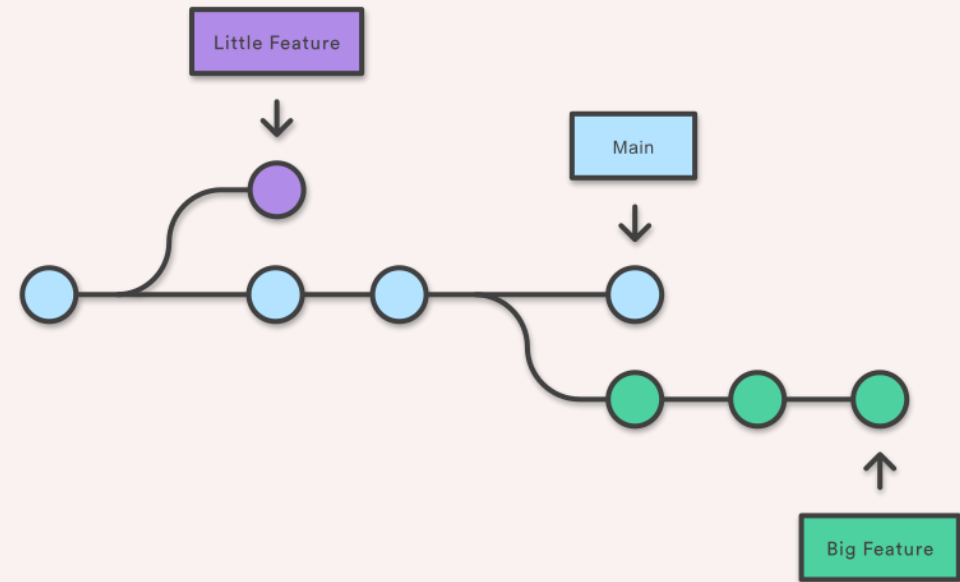
🔗 GitHub Flow

- ງ່າຍດາຍ
- ການປ່ອຍເວີຊັນຕໍ່ເນື່ອງ
- ເໝາະກັບເວັບແອັບ

🔧 Git Flow

- ຊັບຊ້ອນກວ່າ
- ການປ່ອຍເວີຊັນຕາມຕາຕະລາງ
- ເໝາະກັບໂຄງການຂະໜາດໃຫຍ່

ການເລືອກ Workflow ທີ່ເໝາະສົມ



ການເລືອກ Workflow ຂຶ້ນຢູ່ກັບລັກສະນະຂອງໂຄງການ, ຂະໜາດຂອງທີມ, ແລະ ນະໂຍບາຍການປ່ອຍຜະລິດຕະພັນ

GitHub Flow (The Simple Workflow)

Workflow ທີ່ງ່າຍດາຍ ແລະ ຖືກນິຍົມໃຊ້ໂດຍ GitHub ເອງ

- 1 **ສະພາບໃນ main branch**
ຕ້ອງຢູ່ໃນສະພາບ "Deploy ໄດ້ສະເໝີ"
- 2 **ສ້າງ Branch ໃໝ່**
ຈາກ main branch ເມື່ອຕ້ອງການເຮັດວຽກໃໝ່
- 3 **ເຮັດວຽກ ແລະ Push**
Commit ແລະ Push ໃສ່ Branch ຂອງທ່ານຢ່າງສະໝໍ່າສະເໝີ
- 4 **ເປີດ Pull Request**
ເມື່ອພ້ອມແລ້ວ, ເປີດ Pull Request
- 5 **ທົບທວນ ແລະ ລວມ**
ຫຼັງຈາກທົບທວນ ແລະ ອະນຸມັດ, Merge ເຂົ້າ main
- 6 **Deploy ທັນທີ**
ເມື່ອ Merge ເຂົ້າ main ແລ້ວ, ຄວນຈະມີການ Deploy ທັນທີ

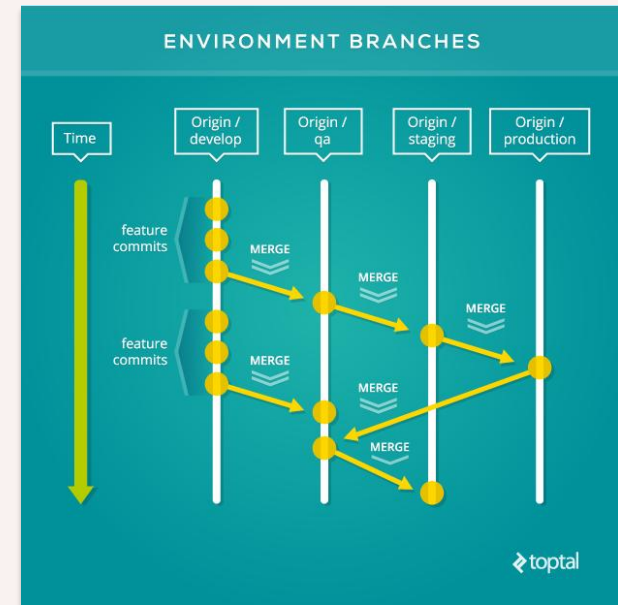
✓ ເໝາະກັບ:

👥 ທີມຂະໜາດນ້ອຍ-ປານກາງ

🔧 ການປ່ອຍເວີຊັນຕໍ່ເນື່ອງ

🌐 ເວັບແອັບພລິເຄຊັນ

GitHub Flow



GitHub Flow ເປັນ Workflow ທີ່ງ່າຍດາຍ, ເນັ້ນການສ້າງ Branch, ເປີດ Pull Request, ແລະ Deploy ທັນທີຫຼັງຈາກ Merge

Git Flow (The Structured Workflow)

Workflow ທີ່ຊັບຊ້ອນກວ່າ, ອອກແບບໂດຍ Vincent Driessen

- ❖ **main (or master)**
Branch ສັກສິດ, ເກັບສະເພາະໂຄດຂອງເວີຊັນທີ່ປ່ອຍອອກສູ່ຕະຫຼາດແລ້ວ
- <> **develop**
Branch ຫຼັກຂອງການພັດທະນາ, ລວມ Feature ທີ່ພັດທະນາສໍາເລັດແລ້ວ
- + **feature/***
ສ້າງຈາກ: develop, ໃຊ້ສໍາລັບການພັດທະນາ Feature ໃໝ່
- release/***
ສ້າງຈາກ: develop, ໃຊ້ສໍາລັບການແກ້ໄຂ Bug ນ້ອຍໆ ກ່ອນການປ່ອຍ
- 🔧 **hotfix/***
ສ້າງຈາກ: main, ໃຊ້ສໍາລັບການແກ້ໄຂ Bug ທີ່ຮ້າຍແຮງ ໃນ Production

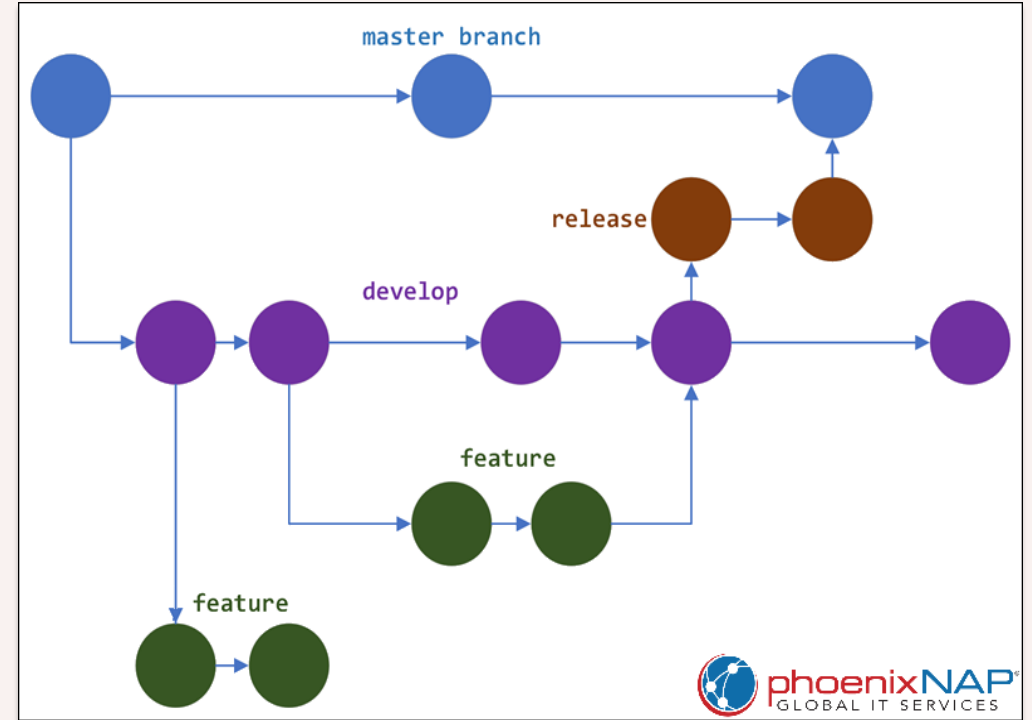
☑ ເໝາະກັບ:

🏢 ໂຄງການຂະໜາດໃຫຍ່

🕒 ການປ່ອຍເວີຊັນຕາມຕາຕະລາງ

🏠 ໂມບາຍ/ເດສກະທັອບແອັບ

Git Flow



Git Flow ເປັນ Workflow ທີ່ມີໂຄງສ້າງຊັດເຈນ, ໃຊ້ Branch ຫຼາຍປະເພດເພື່ອຈັດການການພັດທະນາ, ການ release, ແລະ ການແກ້ໄຂ Bug ສຸກເສີນ

ການປຽບທຽບ GitHub Flow ແລະ Git Flow

🔹 ບົດສະຫຼຸບ

ບໍ່ມີ Workflow ໃດທີ່ **"ດີທີ່ສຸດ"**. ການເລືອກ Workflow ຂຶ້ນຢູ່ກັບລັກສະນະຂອງໂຄງການ, ຂະໜາດຂອງທີມ, ແລະ ນະໂຍບາຍການປ່ອຍຜະລິດຕະພັນ.

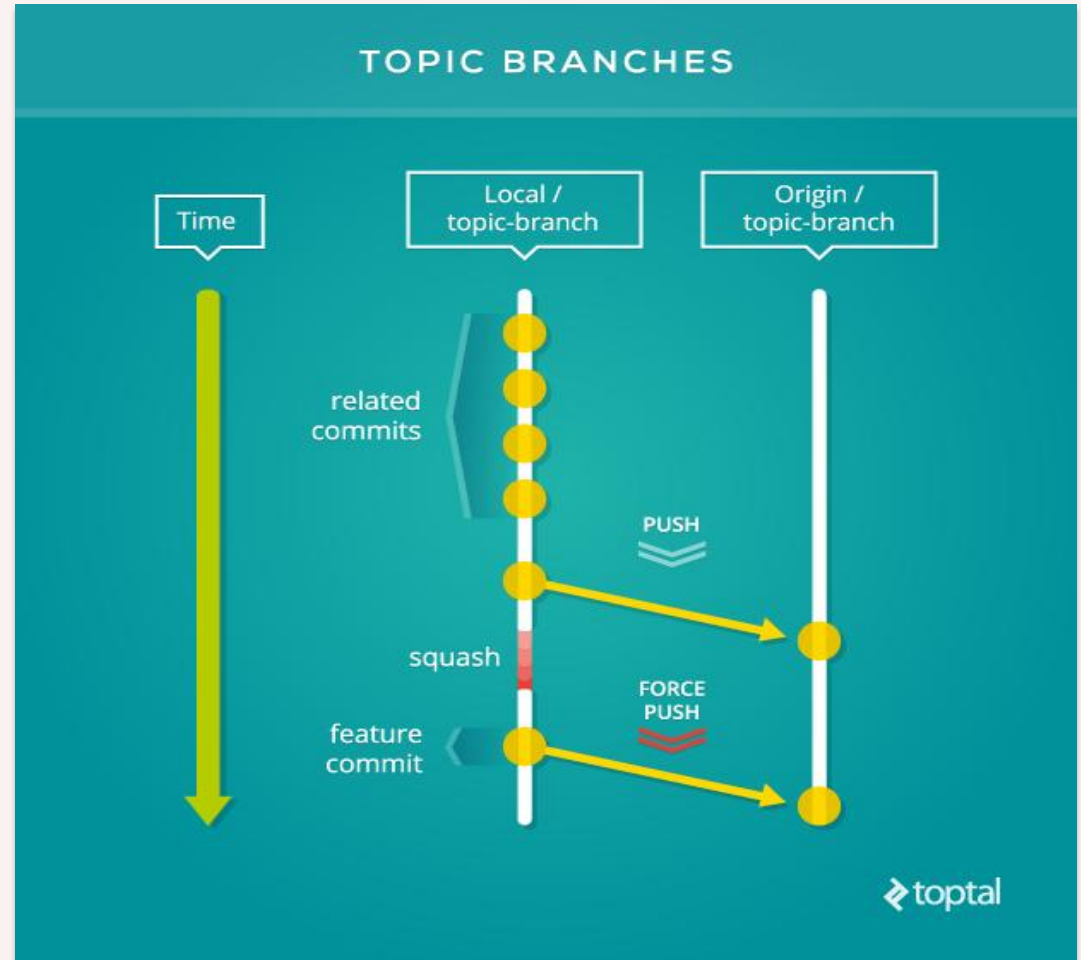
- **GitHub Flow** ເໝາະສໍາລັບໂຄງການສ່ວນໃຫຍ່ ເນື່ອງຈາກຄວາມງ່າຍດາຍຂອງມັນ
- **Git Flow** ເໝາະສໍາລັບໂຄງການທີ່ມີຄວາມຊັບຊ້ອນສູງ ແລະ ຕ້ອງການການຈັດການເວີຊັນທີ່ເຄັ່ງຄັດ
- ສິ່ງທີ່ສໍາຄັນທີ່ສຸດຄືການທີ່ທີມງານທຸກຄົນເຂົ້າໃຈ ແລະ ຕົກລົງທີ່ຈະໃຊ້ Workflow ດຽວກັນ

ບົດສະຫຼຸບ

- ✦ **ທັກສະພື້ນຖານທີ່ຈຳເປັນ**
ການໃຊ້ VCS ບໍ່ແມ່ນ "ທາງເລືອກ" ແຕ່ເປັນ **ທັກສະພື້ນຖານ** ສຳລັບນັກພັດທະນາຊອບແວ
- ⤴ **ປ່ຽນການທຳງານຄົນດຽວໃຫ້ກາຍເປັນມືອາຊີບ**
ຊ່ວຍໃຫ້ການຂຽນໂຄດຄົນດຽວ ກາຍເປັນການພັດທະນາແບບມືອາຊີບທີ່ສາມາດ**ຮ່ວມມືກັນ**ໄດ້
- ✦ **ຮັບປະກັນຄຸນນະພາບ ແລະ ຄວາມໂປ່ງໃສ**
ຮັບປະກັນຄຸນນະພາບ, ສ້າງຄວາມໂປ່ງໃສ, ແລະ ເປັນບ່ອນບັນທຶກ**ປະຫວັດສາດການຕັດສິນໃຈ**

ຂອບໃຈ!
ມີຄຳຖາມບໍ?

ການຮ່ວມມືດ້ວຍ Git



ການເຂົ້າໃຈ ແລະ ໃຊ້ງານ Git ໄດ້ຢ່າງຄ່ອງແຄ້ວ ເປັນທັກສະທີ່ຈຳເປັນທີ່ສຸດໃນການເຮັດວຽກເປັນທີມພັດທະນາຊອບແວ